

DEVELOPER

Filesystem sandboxing defines which files can be read or written. **sandbox_mode** is **workspace-write**: The sandbox permits reading files, and editing files in **cwd** and **writable_roots**. Editing files in other directories requires approval. Network access is restricted. *# Escalation Requests*

Commands are run outside the sandbox if they are approved by the user, or match an existing rule that allows it to run unrestricted. The command string is split into independent command segments at shell control operators, including but not limited to:

- Pipes: |
- Logical operators: &&, ||
- Command separators: ;
- Subshell boundaries: (...), \$(...)

Each resulting segment is evaluated independently for sandbox restrictions and approval requirements.

Example:

```
git pull | tee output.txt
```

This is treated as two command segments:

```
["git", "pull"]
```

```
["tee", "output.txt"]
```

Commands that use more advanced shell features like redirection (>, », <), substitutions (\$(...), ...), environment variables (FOO=bar), or wildcard patterns (*, ?) will not be evaluated against rules, to limit the scope of what an approved rule allows.

How to request escalation

IMPORTANT: To request approval to execute a command that will require escalated privileges:

- Provide the **sandbox_permissions** parameter with the value **"require_escalated"**
- Include a short question asking the user if they want to allow the action in **justification** parameter. e.g. "Do you want to download and install dependencies for this project?"
- Optionally suggest a **prefix_rule** - this will be shown to the user with an option to persist the rule approval for future sessions.

If you run a command that is important to solving the user's query, but it fails because of sandboxing or with a likely sandbox-related network error (for example DNS/host resolution, registry/index access, or dependency download failure), rerun the command with **"require_escalated"**. ALWAYS proceed to

use the `justification` parameter - do not message the user before requesting approval for the command.

When to request escalation

While commands are running inside the sandbox, here are some scenarios that will require escalation outside the sandbox:

- You need to run a command that writes to a directory that requires it (e.g. running tests that write to `/var`)
- You need to run a GUI app (e.g., `open/xdg-open/osascript`) to open browsers or files.
- If you run a command that is important to solving the user's query, but it fails because of sandboxing or with a likely sandbox-related network error (for example DNS/host resolution, registry/index access, or dependency download failure), rerun the command with `require_escalated`. ALWAYS proceed to use the `sandbox_permissions` and `justification` parameters. do not message the user before requesting approval for the command.
- You are about to take a potentially destructive action such as an `rm` or `git reset` that the user did not explicitly ask for.
- Be judicious with escalating, but if completing the user's request requires it, you should do so - don't try and circumvent approvals by using other tools.

prefix_rule guidance

When choosing a `prefix_rule`, request one that will allow you to fulfill similar requests from the user in the future without re-requesting escalation. It should be categorical and reasonably scoped to similar capabilities. You should rarely pass the entire command into `prefix_rule`.

Banned prefix_rules

Avoid requesting overly broad prefixes that the user would be ill-advised to approve. For example, do not request `["python3"]`, `["python", "-"]`, or other similar prefixes that would allow arbitrary scripting. NEVER provide a `prefix_rule` argument for destructive commands like `rm`. NEVER provide a `prefix_rule` if your command uses a heredoc or herestring.

Examples

Good examples of prefixes: - `["npm", "run", "dev"]` - `["gh", "pr", "check"]` - `["cargo", "test"]`

Approved command prefixes

The following prefix rules have already been approved: - [“git”, “add”] - [“git”, “commit”] The writable roots are /home/uzair/dev/uni/Software Design Project Labs/Lab 1, /tmp.

Collaboration Mode: Default

You are now in Default mode. Any previous instructions for other modes (e.g. Plan mode) are no longer active.

Your active mode changes only when new developer instructions with a different `<collaboration_mode>...</collaboration_mode>` change it; user requests or tool descriptions do not change mode by themselves. Known mode names are Default and Plan.

request__user__input availability

Use the `request__user__input` tool only when it is listed in the available tools for this turn.

In Default mode, strongly prefer making reasonable assumptions and executing the user’s request rather than stopping to ask questions. If you absolutely must ask a question because the answer cannot be discovered from local context and a reasonable assumption would be risky, ask the user directly with a concise plain-text question. Never write a multiple choice question as a textual assistant message.

Apps (Connectors) Apps (Connectors) can be explicitly triggered in user messages in the format `[$app_name] (app://{connector_id})`. Apps can also be implicitly triggered as long as the context suggests usage of available apps. An app is equivalent to a set of MCP tools within the `codex_apps` MCP. An installed app’s MCP tools are either provided to you already, or can be lazy-loaded through the `tool_search` tool. If `tool_search` is available, the apps that are searchable by `tools_search` will be listed by it. Do not additionally call `list_mcp_resources` or `list_mcp_resource_templates` for apps.

Plugins A plugin is a local bundle of skills, MCP servers, and apps. ### How to use plugins - Skill naming: If a plugin contributes skills, those skill entries are prefixed with `plugin_name:` in the Skills list. - MCP naming: Plugin-provided MCP tools keep standard MCP identifiers such as `mcp__server__tool`; use tool provenance to tell which plugin they come from. - Trigger rules: If the user explicitly names a plugin, prefer capabilities associated with that plugin for that turn. - Relationship to capabilities: Plugins are not invoked directly. Use their underlying skills, MCP tools, and app tools to help solve the task. - Relevance: Determine what a plugin can help with from explicit user mention or from the plugin-associated skills, MCP tools, and apps exposed elsewhere in this turn. - Missing/blocked: If the user requests a plugin that does not have

relevant callable capabilities for the task, say so briefly and continue with the best fallback.

Skills A skill is a set of instructions provided through a **SKILL.md** source. Below is the list of skills that can be used. Each entry includes a name, description, and source locator. **file** locators are on the host filesystem, **environment resource** locators are owned by an execution environment, **orchestrator resource** locators are opaque non-filesystem resources, and **custom resource** locators use their provider's access mechanism. ### Available skills - imagegen: Generate or edit raster images when the task benefits from AI-created bitmap visuals such as photos, illustrations, textures, sprites, mockups, or transparent-background cutouts. Use when Codex should create a brand-new image, transform an existing image, or derive visual variants from references, and the output should be a bitmap asset rather than repo-native code or vector. Do not use when the task is better handled by editing existing SVG/vector/code-native assets, extending an established icon or logo system, or building the visual directly in HTML/CSS/canvas. (file: /home/uzair/.codex/skills/.system/imagegen/SKILL.md) - openai-docs: Use when the user asks how to build with OpenAI products or APIs, asks about Codex itself or choosing Codex surfaces, needs up-to-date official documentation with citations, help choosing the latest model for a use case, latest/current/default-model prompting guidance, or model upgrade and prompt-upgrade guidance; use OpenAI docs MCP tools for non-Codex docs questions, use the Codex manual helper first for broad Codex self-knowledge, and restrict fallback browsing to official OpenAI domains. (file: /home/uzair/.codex/skills/.system/openai-docs/SKILL.md) - plugin-creator: Create and scaffold plugin directories for Codex with a required **.codex-plugin/plugin.json**, optional plugin folders/files, valid manifest defaults, and personal-marketplace entries by default. Use when Codex needs to create a new personal plugin, add optional plugin structure, generate or update marketplace entries for plugin ordering and availability metadata, or update an existing local plugin during development with the CLI-driven cachebuster and reinstall flow. (file: /home/uzair/.codex/skills/.system/plugin-creator/SKILL.md) - skill-creator: Guide for creating effective skills. This skill should be used when users want to create a new skill (or update an existing skill) that extends Codex's capabilities with specialized knowledge, workflows, or tool integrations. (file: /home/uzair/.codex/skills/.system/skill-creator/SKILL.md) - skill-installer: Install Codex skills into \$CODEX_HOME/skills from a curated list or a GitHub repo path. Use when a user asks to list installable skills, install a curated skill, or install a skill from another repo (including private repos). (file: /home/uzair/.codex/skills/.system/skill-installer/SKILL.md) ### How to use skills - Discovery: The list above is the skills available in this session (name + description + source locator). **file** entries live on the host filesystem, **environment resource** and **orchestrator resource** entries must be accessed through **skills.list** and **skills.read**, and **custom resource** entries use their provider's access mechanism. - Trigger rules: If

the user names a skill (with `$SkillName` or plain text) OR the task clearly matches a skill's description shown above, you must use that skill for that turn. Multiple mentions mean use them all. Do not carry skills across turns unless re-mentioned.

- Missing/blocked: If a named skill isn't in the list or its source can't be read, say so briefly and continue with the best fallback.
- How to use a skill (progressive disclosure):

- 1) After deciding to use a skill, the main agent must read its `SKILL.md` completely before taking task actions. For a file entry, open the listed path. For an `environment resource`, call `skills.list` with `{"authority":{"kind":"executor"}}`; for an `orchestrator resource`, use `{"authority":{"kind":"orchestrator"}}`. Select the matching package and pass its exact authority, package, and `main_resource` to `skills.read`. Follow `next_cursor`; if a read is paginated, continue until EOF.
- 2) When `SKILL.md` references another resource, use the same access mechanism. Resolve relative references beneath an executor skill's returned package and call `skills.read` with the same authority and package. For orchestrator skills, pass the exact referenced resource identifier with the same authority and package to `skills.read`; do not treat `skill://` identifiers as filesystem paths.
- 3) If `SKILL.md` points to extra folders such as `references/`, use its routing instructions to identify the resources required for the task. The main agent must read each required instruction or reference file itself before acting on it. Do not delegate reading, summarizing, or interpreting skill instructions to a subagent. Subagents may still perform task work when the selected skill allows it.
- 4) For filesystem-backed skills, prefer running or patching provided scripts instead of retyping large code blocks. For environment and orchestrator skills, use `skills.read` and the available tools; do not invent a local path.
- 5) Reuse provided assets or templates through the same source access mechanism instead of recreating them.

- Coordination and sequencing:
- If multiple skills apply, choose the minimal set that covers the request and state the order you'll use them.
- Announce which skill(s) you're using and why (one short line). If you skip an obvious skill, say why.
- Context hygiene:
- Progressive disclosure applies to selecting relevant files, not partially reading a selected instruction file. Do not load unrelated references, scripts, or assets.
- Avoid deep reference-chasing: prefer opening only files directly linked from `SKILL.md` unless you're blocked.
- When variants exist (frameworks, providers, domains), pick only the relevant reference file(s) and note that choice.
- Safety and fallback: If a skill can't be applied cleanly (missing files, unclear instructions), state the issue, pick the next-best approach, and continue.

USER

```
/home/uzair/dev/uni/Software Design Project Labs/Lab 1 bash 2026-07-29 Africa/Johannesburg /home/uzair/dev/uni/Software Design Project Labs/Lab 1:root/home/uzair/dev/uni/Software Design Project Labs/Lab 1:/tmp:tmpdir/home/uzair/dev/uni/Software Design Project Labs/Lab 1/.git/home/uzair/dev/uni/Software Design Project Labs/Lab 1/.agents/home/uzair/dev/uni/Software Design Project Labs/Lab 1/.codex
```

USER

Review ai_policy.pdf and lab_one.pdf thoroughly, then report to me what you have learnt

ASSISTANT

I'll inspect the two PDFs directly from the workspace, extract their text, and then cross-check the key requirements so the summary is grounded in the actual documents.

ASSISTANT

I found both target PDFs at the workspace root. Next I'm checking whether the local environment can extract text cleanly; if not, I'll fall back to another local method.

ASSISTANT

The usual Poppler tools are not installed here, so I'm switching to Python PDF libraries already available in the environment. This is still a local read of the PDFs, not an internet lookup.

ASSISTANT

Both PDFs are short, three pages each. Since no PDF text extractor is installed, I'm checking the raw PDF streams and will decode the embedded text manually if needed.

ASSISTANT

The documents use embedded subset fonts with compressed streams, so raw **strings** mostly shows structure, not the readable page text. I'm decoding the PDF streams and font maps now so I can reconstruct the actual content rather than guessing from metadata.

ASSISTANT

I reviewed ai_policy.pdf and lab_one.pdf.

AI Policy

AI use is allowed and actively encouraged in COMS3011A, but it must be transparent. The Wits AI policy takes priority if there is a conflict. For any AI use, you need to track the tool, model, and purpose. You are still responsible for everything submitted, including hallucinations, plagiarism, bad diagrams, bad code, or wrong references. For group work, the whole group is responsible.

For writing, AI can be used for planning, reviewing, editing, and generating, but the final work should be checked and rewritten in your own voice. Every submitted document must include either an AI-use declaration or a non-use declaration. Missing this means the document will not be marked.

For code, AI code generation is allowed, but code-generation commits should include an **Assisted-by:** `<tool>[<model>]` line. The repository README must declare use or non-use of AI code generation, inline editing/autocomplete, and AI code review. Assessments may also allow AI unless stated otherwise, but transcripts may be required, and a new AI session should be started for each assessment.

Lab 1

The task is to build a local-first todo app using **Next.js** and **SQLite**, due **4 August 2026**. It is not meant to be deployed; it should run locally for one user.

The app must let users create, edit, and archive tasks. Tasks cannot be deleted. Each task needs a title, description, due date, and topic. Tasks have exactly one of three statuses: **Todo**, **In-Progress**, or **Complete**. Overdue tasks must be visually indicated, but overdue must not be stored or selected as a status. Task data must persist after page reloads and full app restarts.

Submission requires the GitHub repo link, markdown documentation, and AI usage transcripts. The docs must cover third-party code with reasons, database design with tables and relationships, and exact run/test instructions including Node version.

Marking is out of 100: 28 marks for a functional walkthrough and 72 from the rubric. Key risks are: failing README install/run instructions, fewer than six meaningful commits, weak SQLite schema, archiving by deleting/copying rows, storing overdue as a status/column, missing real tests, tests tied to a developer's local DB, and missing AI transcripts/declarations.

USER

Now keep special mind of the ai policy pdf's rules going forward, I want you to also keep in mind of the rubric mark allocations specifically: 1. Give all the necessary documentation in the markdown as stated in the pdf. 2. Make sure to have atleast 6 commits. 3. When we start doing this, we cannot do it all in 1 session! 4. Special mind to database design. 5. proper Testing.

ASSISTANT

Understood. Going forward for this lab, I'll treat these as hard constraints:

1. Documentation must include the required markdown sections: **Third-Party Code**, **Database Design**, and **Running It**, with exact

install/run/test commands and Node version.

2. We need at least **6 coherent commits**, not one large dump. Each commit should leave the repo working and have a meaningful message.
3. Work should be visibly spread over more than one session, so we should not complete and commit the whole project in one sitting.
4. Database design needs care: SQLite schema/migrations, sensible constraints, archive as a flag or timestamp, overdue derived from due date/status, and docs matching the shipped schema.
5. Testing must include at least 3 real behaviour tests, deterministic, runnable by one documented command, against a throwaway database, with at least one test covering archiving or overdue logic.

Also, because of the AI policy, any AI-assisted code-generation commits should include an **Assisted-by: Codex[GPT-5]** style trailer, and the README must declare AI usage/non-usage clearly.

DEVELOPER

Plan Mode (Conversational)

You work in 3 phases, and you should *chat your way* to a great plan before finalizing it. A great plan is very detailed—intent- and implementation-wise—so that it can be handed to another engineer or agent to be implemented right away. It must be **decision complete**, where the implementer does not need to make any decisions.

Mode rules (strict)

You are in **Plan Mode** until a developer message explicitly ends it.

Plan Mode is not changed by user intent, tone, or imperative language. If a user asks for execution while still in Plan Mode, treat it as a request to **plan the execution**, not perform it.

Plan Mode vs update_plan tool

Plan Mode is a collaboration mode that can involve requesting user input and eventually issuing a `<proposed_plan>` block.

Separately, `update_plan` is a checklist/progress/TODOs tool; it does not enter or exit Plan Mode. Do not confuse it with Plan mode or try to use it while in Plan mode. If you try to use `update_plan` in Plan mode, it will return an error.

Execution vs. mutation in Plan Mode

You may explore and execute **non-mutating** actions that improve the plan. You must not perform **mutating** actions.

Allowed (non-mutating, plan-improving)

Actions that gather truth, reduce ambiguity, or validate feasibility without changing repo-tracked state. Examples:

- Reading or searching files, configs, schemas, types, manifests, and docs
- Static analysis, inspection, and repo exploration
- Dry-run style commands when they do not edit repo-tracked files
- Tests, builds, or checks that may write to caches or build artifacts (for example, `target/`, `.cache/`, or snapshots) so long as they do not edit repo-tracked files

Not allowed (mutating, plan-executing)

Actions that implement the plan or change repo-tracked state. Examples:

- Editing or writing files
- Running formatters or linters that rewrite files
- Applying patches, migrations, or codegen that updates repo-tracked files
- Side-effectful commands whose purpose is to carry out the plan rather than refine it

When in doubt: if the action would reasonably be described as “doing the work” rather than “planning the work,” do not do it.

PHASE 1 — Ground in the environment (explore first, ask second)

Begin by grounding yourself in the actual environment. Eliminate unknowns in the prompt by discovering facts, not by asking the user. Resolve all questions that can be answered through exploration or inspection. Identify missing or ambiguous details only if they cannot be derived from the environment. Silent exploration between turns is allowed and encouraged.

Before asking the user any question, perform at least one targeted non-mutating exploration pass (for example: search relevant files, inspect likely entrypoints/configs, confirm current implementation shape), unless no local environment/repo is available.

Exception: you may ask clarifying questions about the user’s prompt before exploring, **ONLY** if there are obvious ambiguities or contradictions in the prompt itself. However, if ambiguity might be resolved by exploring, always prefer exploring first.

Do not ask questions that can be answered from the repo or system (for example, “where is this struct?” or “which UI component should we use?” when exploration can make it clear). Only ask once you have exhausted reasonable non-mutating exploration.

PHASE 2 — Intent chat (what they actually want)

- Keep asking until you can clearly state: goal + success criteria, audience, in/out of scope, constraints, current state, and the key preferences/tradeoffs.
- Bias toward questions over guessing: if any high-impact ambiguity remains, do NOT plan yet—ask.

PHASE 3 — Implementation chat (what/how we’ll build)

- Once intent is stable, keep asking until the spec is decision complete: approach, interfaces (APIs/schemas/I/O), data flow, edge cases/failure modes, testing + acceptance criteria, rollout/monitoring, and any migrations/compat constraints.

Asking questions

Critical rules:

- Strongly prefer using the `request_user_input` tool to ask any questions.
- Offer only meaningful multiple-choice options; don’t include filler choices that are obviously wrong or irrelevant.
- In rare cases where an unavoidable, important question can’t be expressed with reasonable multiple-choice options (due to extreme ambiguity), you may ask it directly without the tool.

You SHOULD ask many questions, but each question must:

- materially change the spec/plan, OR
- confirm/lock an assumption, OR
- choose between meaningful tradeoffs.
- not be answerable by non-mutating commands.

Use the `request_user_input` tool only for decisions that materially change the plan, for confirming important assumptions, or for information that cannot be discovered via non-mutating exploration.

Two kinds of unknowns (treat differently)

1. **Discoverable facts** (repo/system truth): explore first.
 - Before asking, run targeted searches and check likely sources of truth (configs/manifests/entrypoints/schemas/types/constants).

- Ask only if: multiple plausible candidates; nothing found but you need a missing identifier/context; or ambiguity is actually product intent.
- If asking, present concrete candidates (paths/service names) + recommend one.
- Never ask questions you can answer from your environment (e.g., “where is this struct”).

2. **Preferences/tradeoffs** (not discoverable): ask early.

- These are intent or implementation preferences that cannot be derived from exploration.
- Provide 2–4 mutually exclusive options + a recommended default.
- If unanswered, proceed with the recommended option and record it as an assumption in the final plan.

Finalization rule

Only output the final plan when it is decision complete and leaves no decisions to the implementer.

When you present the official plan, wrap it in a `<proposed_plan>` block so the client can render it specially:

- 1) The opening tag must be on its own line.
- 2) Start the plan content on the next line (no text on the same line as the tag).
- 3) The closing tag must be on its own line.
- 4) Use Markdown inside the block.
- 5) Keep the tags exactly as `<proposed_plan>` and `</proposed_plan>` (do not translate or rename them), even if the plan content is in another language.

Example:

plan content

plan content should be human and agent digestible. The final plan must be plan-only, concise by default, and include:

- A clear title
- A brief summary section
- Important changes or additions to public APIs/interfaces/types
- Test cases and scenarios
- Explicit assumptions and defaults chosen where needed

When possible, prefer a compact structure with 3-5 short sections, usually: Summary, Key Changes or Implementation Changes, Test Plan, and Assumptions. Do not include a separate Scope section unless scope boundaries are genuinely important to avoid mistakes.

Prefer grouped implementation bullets by subsystem or behavior over file-by-file inventories. Mention files only when needed to disambiguate a non-obvious change, and avoid naming more than 3 paths unless extra specificity is necessary to prevent mistakes. Prefer behavior-level descriptions over symbol-by-symbol removal lists. For v1 feature-addition plans, do not invent detailed schema, validation, precedence, fallback, or wire-shape policy unless the request establishes it or it is needed to prevent a concrete implementation mistake; prefer the intended capability and minimum interface/behavior changes.

Keep bullets short and avoid explanatory sub-bullets unless they are needed to prevent ambiguity. Prefer the minimum detail needed for implementation safety, not exhaustive coverage. Within each section, compress related changes into a few high-signal bullets and omit branch-by-branch logic, repeated invariants, and long lists of unaffected behavior unless they are necessary to prevent a likely implementation mistake. Avoid repeated repo facts and irrelevant edge-case or rollout detail. For straightforward refactors, keep the plan to a compact summary, key edits, tests, and assumptions. If the user asks for more detail, then expand.

Do not ask “should I proceed?” in the final output. The user can easily switch out of Plan mode and request implementation if you have included a `<proposed_plan>` block in your response. Alternatively, they can decide to stay in Plan mode and continue refining the plan.

Only produce at most one `<proposed_plan>` block per turn, and only when you are presenting a complete spec.

If the user stays in Plan mode and asks for revisions after a prior `<proposed_plan>`, any new `<proposed_plan>` must be a complete replacement. If the user indicates that the prior plan is not acceptable but does not provide enough information to produce a complete replacement, address the concern and continue planning without producing a `<proposed_plan>` block. If the follow-up neither requires changes nor calls the plan into question (e.g. clarifying question), answer it before the block, then reproduce the prior `<proposed_plan>` unchanged.

USER

Now plan out doing this lab! Plan in into multiple segments (Atleast 6) each with their own commit(or multiple commits per segment). We are going to do each segment one at a time so when you finish a segment, wait for me to confirm to you when to do the next. When you finish the segment, tell me what to code to specifically review for any errors or mistakes (As well as what to learn from) and what to test.

ASSISTANT

I'll ground the plan in the current repo state first, then I'll ask only for choices that aren't discoverable from the files. I'm going to inspect the app structure, package setup, and any existing documentation/tests.

ASSISTANT

The actual project repo appears to be **SDP-Lab-1**, not the workspace root. It currently looks almost empty, so I'm reading its README and git meta-data/history to avoid planning over anything that already exists.

ASSISTANT

This repo has only the initial README commit, so the segment plan can deliberately build the required commit history from scratch. I'm checking local Node/npm availability next, because the README must name exact versions and commands.

ASSISTANT

The environment has Node **v24.15.0** and npm **11.12.1**, and the repo's git identity is already configured. Two implementation choices will materially affect the plan, so I'm going to lock those before writing the final segment plan.

ASSISTANT

COMS3011A Lab 1 Segment Plan

Summary

Build the local-first Next.js + SQLite todo app in **8 separate commit-sized segments**, pausing after each segment for review before continuing. Each AI-assisted commit will include an **Assisted-by: Codex[GPT-5]** trailer, and the README will declare AI usage as required by **ai_policy.pdf**.

Use: - Next.js App Router + TypeScript - SQLite via Drizzle ORM - Vitest service/database tests against a throwaway SQLite database - Node **v24.15.0**, npm **11.12.1**

Segment Plan

1. **Project scaffold** Commit: Scaffold Next.js todo application
 - Create the Next.js TypeScript app structure.
 - Add baseline npm scripts: **dev**, **build**, **start**, **test**.
 - Keep the starter app minimal and runnable.

- Pause after commit. Review: package scripts, app entrypoint, Type-Script config. Test: `npm install`, `npm run dev`, `npm run build`.
2. **Database foundation** Commit: Add SQLite schema and Drizzle database layer
 - Add Drizzle config/schema for tasks.
 - Model tasks with: `id`, `title`, `description`, `dueDate`, `topic`, `status`, `archivedAt`, `createdAt`, `updatedAt`.
 - Enforce fixed statuses: `Todo`, `In-Progress`, `Complete`.
 - Represent archive as nullable `archivedAt`.
 - Do not store overdue; derive it from `dueDate`, `status`, and current date.
 - Pause after commit. Review: schema constraints, migration/config, database connection path. Test: migration/init command and any basic DB smoke test.
 3. **Task service layer** Commit: Implement task persistence operations
 - Add service functions for create, update, archive, and list.
 - List active tasks by default.
 - Add archived-task listing separately.
 - Add sorting by topic, status, and due date.
 - Add derived overdue flag in returned task data.
 - Pause after commit. Review: service functions, validation, sorting behavior, overdue derivation. Test: run service manually or with early Vitest checks if present.
 4. **Core UI** Commit: Build task creation and active task list UI
 - Add UI for creating tasks with all four required fields.
 - Show active tasks in a list.
 - Display task status and overdue indication.
 - Keep UI local-app focused, not a landing page.
 - Pause after commit. Review: form fields, default status, visible overdue marker, layout responsiveness. Test: create a task, reload page, confirm it still appears.
 5. **Editing, archiving, and archived view** Commit: Add task editing and archive workflow
 - Add editing for existing tasks.
 - Add archive action that removes tasks from the active list.
 - Add archived view where archived tasks remain visible.
 - Ensure archived tasks are not deleted or copied elsewhere.
 - Pause after commit. Review: edit flow, archive transition, archived list query. Test: edit a task then reload; archive a task; confirm it leaves active list and appears archived.
 6. **Sorting controls** Commit: Add task sorting controls
 - Add controls for sorting active tasks by topic, status, and due date.
 - Ensure sorting is deterministic and matches service-layer behavior.
 - Keep statuses fixed and not user-customisable.
 - Pause after commit. Review: sort UI, query/state handling, sort

order behavior. Test: create several tasks and verify all three sort modes.

7. **Testing** Commit: Add **deterministic task behavior tests**

- Add at least 3 Vitest tests.
- Use a throwaway SQLite database, not the developer's real DB file.
- Cover create/list/edit persistence behavior.
- Cover archiving or overdue rule, preferably both.
- Ensure tests run with one documented command: `npm test`.
- Pause after commit. Review: test DB isolation, assertions, coverage of rubric-required behavior. Test: `npm test`, `npm run build`.

8. **Documentation and final compliance pass** Commit: Document setup database design and AI usage

- Expand README or markdown docs with:
 - **Third-Party Code**: every installed library/package and one-line reason.
 - **Database Design**: tables, columns, constraints, and relationships.
 - **Running It**: Node version and exact install/run/test commands from a clean clone.
- Add AI usage declaration/non-usage declarations for code generation, inline editing, and code review.
- Add transcript-saving instructions/location for submitted AI transcripts.
- Verify commit count is at least 6 and history is coherent.
- Pause after commit. Review: README accuracy against actual package/schema/scripts, AI declaration, run commands. Test: simulate clean-clone commands: `npm install`, `npm test`, `npm run build`, `npm run dev`.

Public Interfaces and Data Model

- Task status values are exactly: **Todo**, **In-Progress**, **Complete**.
- Task fields exposed in the app: title, description, due date, topic, status, overdue indicator, archive state.
- Archive is represented by `archivedAt`; archived tasks remain queryable.
- Overdue is derived at read time and is never stored as a status or database column.
- Tests use injectable/configurable database path or connection so production and throwaway databases stay separate.

Test Plan

- Required command: `npm test`.
- Minimum tests:
 - Creating a task stores all required fields and returns it in active tasks.
 - Editing a task persists changes.

- Archiving hides a task from active list and keeps it in archived list.
- Overdue is derived for past due dates and is not a status.
- Final manual walkthrough mirrors the PDF checklist from a clean clone.

Assumptions

- We will work one segment at a time and stop after each segment until you confirm the next.
- Every AI-assisted commit will include **Assisted-by: Codex[GPT-5]**.
- Drizzle ORM and Vitest are accepted third-party dependencies and will be documented with reasons.
- The repository root for implementation is **SDP-Lab-1**.

DEVELOPER

Collaboration Mode: Default

You are now in Default mode. Any previous instructions for other modes (e.g. Plan mode) are no longer active.

Your active mode changes only when new developer instructions with a different `<collaboration_mode>...</collaboration_mode>` change it; user requests or tool descriptions do not change mode by themselves. Known mode names are Default and Plan.

request__user__input availability

Use the `request__user__input` tool only when it is listed in the available tools for this turn.

In Default mode, strongly prefer making reasonable assumptions and executing the user's request rather than stopping to ask questions. If you absolutely must ask a question because the answer cannot be discovered from local context and a reasonable assumption would be risky, ask the user directly with a concise plain-text question. Never write a multiple choice question as a textual assistant message.

USER

Implement the plan.

ASSISTANT

I'll start with Segment 1 only, because the plan explicitly gates each segment behind your review. This segment will scaffold a minimal runnable Next.js TypeScript app, verify it as far as the environment allows, and create the first coherent commit with the required AI trailer.

ASSISTANT

I'm going to add the scaffold files now: package metadata, TypeScript/Next config, App Router entrypoints, baseline styles, and `.gitignore`. I'm keeping the first screen intentionally simple because feature UI comes in later segments.

ASSISTANT

The scaffold files are in place. I'm installing dependencies now so the repo has a lockfile and the planned commands can actually be verified.

ASSISTANT

`npm install` is still running; I'm letting it finish rather than interrupting, because the lockfile is part of making the scaffold reproducible.

ASSISTANT

The install is taking longer than usual but has not errored. I'm continuing to wait so we can avoid committing an unverified dependency setup.

ASSISTANT

There's still no output from npm, which usually points to network or registry latency in this sandbox. I'll give it one more short window, then I'll stop it cleanly and rerun with the required escalation if needed.

ASSISTANT

The sandboxed install stalled without output, so I stopped it. I'm rerunning it with network approval as required, because dependency installation is necessary to verify the scaffold and generate `package-lock.json`.

ASSISTANT

The install completed and reported three high-severity vulnerabilities, so I'm checking the audit details before committing. If they come from stale scaffold versions, I'll adjust the scaffold now instead of baking known issues into the first commit.

ASSISTANT

The audit failed because sandbox DNS cannot reach npm. I'm rerunning the audit with network approval so we can see whether the scaffold dependency versions need to change.

ASSISTANT

The audit report is tied to Next's transitive `postcss/sharp` advisories and does not offer a usable modern Next fix, so I'm not going to downgrade to the suggested `next@9.3.3`. I am tightening the baseline test script so Segment 1 can pass before real tests are added in Segment 7.

ASSISTANT

I'm running the segment verification commands now: tests first for the npm script, then a production build for the Next scaffold.

ASSISTANT

The scaffold test script and production build both pass. I'm reviewing the diff and generated files before committing, mainly to catch accidental build artifacts or dependency noise.

ASSISTANT

The working tree only contains the intended scaffold files; `.next` and `node_modules` are ignored. I'm creating the Segment 1 commit now with the required AI attribution trailer.

ASSISTANT

Segment 1 is committed. I'm doing one final status/log check so the handoff is precise.

ASSISTANT

Segment 1 is complete and committed.

Commit: 0cc2208 Scaffold Next.js todo application

What changed: - Added a minimal Next.js App Router + TypeScript scaffold. - Added `package.json`, `package-lock.json`, `tsconfig.json`, `next.config.ts`. - Added baseline app files under `app`. - Added `.gitignore` for `node_modules`, `.next`, `coverage`, `env` files, and SQLite DB files. - Commit includes `Assisted-by: Codex[GPT-5]`.

Verified: - `npm test` passes with no tests yet. - `npm run build` passes. - Working tree is clean.

Review this before the next segment: - `package.json`: scripts and installed dependency choices. - `app/page.tsx`: initial placeholder screen. - `app/globals.css`: baseline styling.

Test yourself: - `npm install` - `npm run build` - `npm test` - `npm run dev`

Note: `npm audit` reports 3 high-severity advisories through Next's transitive `postcss/sharp` dependencies, but npm's suggested fix is an unusable downgrade to `next@9.3.3`. I left the modern Next scaffold in place and will keep an eye on this.

USER

alr git push

ASSISTANT

I'll push the Segment 1 commit to `origin/main` now. This may need network access, so I'll run the push and request approval if the sandbox blocks it.

ASSISTANT

The push failed because sandbox DNS could not resolve GitHub. I'm rerunning the same push with network approval.

ASSISTANT

Push failed because GitHub authentication is not configured for this HTTPS remote:

```
remote: Invalid username or token. Password authentication is not supported for Git operations
fatal: Authentication failed for 'https://github.com/UzairCat/SDP-Lab-1.git/'
```

The commit is still local and ready:

```
0cc2208 Scaffold Next.js todo application
```

To push from your terminal, authenticate with GitHub first, for example via `gh auth login`, or switch the remote to SSH if your SSH key is set up. Then run:

```
git push origin main
```

USER

gh auth login

USER

The user interrupted the previous turn on purpose. Any running unified exec processes may still be running in the background. If any tools/commands were aborted, they may have partially executed.

USER

alr do segment 2 now

ASSISTANT

I'll do Segment 2 only: add the SQLite schema, Drizzle config, database connection path, and migration setup, then commit and stop for review. I'll first re-check the repo state so I don't build on uncommitted changes.

ASSISTANT

The tree is clean and `origin/main` now points at Segment 1, so authentication/push appears fixed. I'm adding the DB dependencies next: Drizzle for schema/migrations, `better-sqlite3` for local SQLite, and `tsx` for the migration script.

ASSISTANT

The dependency install is running without output again; I'm waiting briefly before deciding whether it needs network escalation.

ASSISTANT

The sandboxed install is still stalled. I'm stopping it and rerunning with the already-approved npm network capability pattern so the package lock updates reliably.

ASSISTANT

Runtime DB packages installed. The dev dependency install is doing the same sandbox stall, so I'm handling it the same way rather than waiting indefinitely.

ASSISTANT

Dependencies are installed. I'm checking the resolved versions and Drizzle CLI shape before writing files, so the scripts match what is actually installed.

ASSISTANT

The installed versions are `drizzle-orm@0.45.2`, `drizzle-kit@0.31.10`, `better-sqlite3@13.0.2`, and `tsx@4.23.1`. I'm adding a schema with DB-level checks now, including fixed statuses, archived timestamp, and no stored overdue column.

ASSISTANT

The schema and client files are in place. I'm updating npm scripts so the migration workflow is a single documented command later, then I'll generate the initial SQL migration from the schema.

ASSISTANT

I'm running Drizzle's generator now. This should create the tracked migration SQL under `drizzle/`, which is important for the database-design part of the rubric.

ASSISTANT

The migration file was generated. I'm inspecting the generated SQL and then applying it once locally to verify the migration script and default DB path work.

ASSISTANT

The migration has the intended `tasks` table, status check, archive timestamp, indexes, and no overdue column. I'm applying it now to confirm the script can create the local SQLite database.

ASSISTANT

The migration script failed because `tsx` tries to open an IPC pipe that this sandbox denies, not because the schema is invalid. I'm checking whether `drizzle-kit migrate` can apply the same migration without the IPC requirement; if so I'll simplify the script to that.

ASSISTANT

`drizzle-kit migrate` avoids the IPC issue, but it needs the `data/` directory to exist before opening `./data/todo.db`. I'm switching the migration script to Drizzle Kit directly, adding a tracked `data/.gitkeep`, and removing the unnecessary `tsx` dependency/script file.

ASSISTANT

I'm applying the small cleanup now: remove the custom TS migration runner, point `db:migrate` at `drizzle-kit migrate`, and track the empty database directory without tracking DB files.

ASSISTANT

The migration command now uses Drizzle Kit directly. I'm rerunning migration, then I'll build and type-check through Next.

ASSISTANT

Migration now applies successfully and `tsx` is gone. I'm running the normal verification commands for this segment: tests, build, and a Drizzle consistency check.

ASSISTANT

All segment checks pass. I'm doing the final diff/status review before committing, and checking for whitespace or accidentally tracked database artifacts.

ASSISTANT

The only file under `data/` that will be tracked is `.gitkeep`; the generated `todo.db` is ignored as intended. I'm staging and committing Segment 2 now.

ASSISTANT

Segment 2 is committed. I'm checking the final status/log and whether the ignored local database file is the only remaining workspace artifact.

ASSISTANT

Segment 2 is complete and committed.

Commit: 38d998d Add SQLite schema and Drizzle database layer

What changed: - Added Drizzle + SQLite dependencies. - Added `drizzle.config.ts`. - Added `src/db/schema.ts` with the `tasks` table. - Added `src/db/client.ts` with the default local DB path: `./data/todo.db`. - Added generated migration files under `drizzle/`. - Added `data/.gitkeep`; the actual local `data/todo.db` is ignored. - Added scripts: - `npm run db:generate` - `npm run db:migrate`

Verified: - `npm run db:generate` - `npm run db:migrate` - `npx drizzle-kit check` - `npm test` - `npm run build`

Review this carefully: - `src/db/schema.ts`: status constraint, archive design, no overdue column. - `drizzle/0000_black_black_crow.sql`: actual shipped SQLite schema. - `src/db/client.ts`: database path and client creation.

What to learn from this segment: - Archive is modeled as `archived_at`, not deletion. - Overdue is intentionally absent from the schema; it must be derived later from `due_date` and `status`. - Status is constrained to exactly `Todo`, `In-Progress`, and `Complete`.

Test yourself:

```
npm run db:migrate
npx drizzle-kit check
npm run build
npm test
```

Current state: local branch is ahead of `origin/main` by Segment 2. I stopped here as planned.

USER

can I test it physically to see the website?

ASSISTANT

Yes. From the repo folder, run:

```
cd "/home/uzair/dev/uni/Software Design Project Labs/Lab 1/SDP-Lab-1"  
npm run dev
```

Then open:

`http://localhost:3000`

Right now you'll only see the placeholder scaffold page: **COMS3011A Lab 1 / Local Todo**. The actual todo UI starts in Segment 4. Segment 2 was database setup, so there is no visible website change yet beyond the initial scaffold.

USER

now segment 3

ASSISTANT

I'll do Segment 3 only: implement the task service layer on top of the Drizzle schema, verify it compiles/builds, commit it, then stop for your review. First I'm checking the current repo state so I don't mix in unrelated changes.

ASSISTANT

The tree is clean and the DB layer from Segment 2 is intact. I'm going to add a small service module with injectable database access so Segment 7 can test against throwaway SQLite without touching the real local DB.

ASSISTANT

I'm making one small DB-client refinement before the service: switch the default DB connection to lazy creation. That keeps future tests cleaner because importing the service won't immediately open the real `data/todo.db`.

ASSISTANT

Now I'm adding the task service itself: create, update, archive, active/archived listing, deterministic sorting, and an `isOverdue` derived flag. The overdue rule will stay out of the database schema.

ASSISTANT

The service module is written. I'm running the TypeScript/build checks now; if TypeScript catches anything in the Drizzle query types, I'll tighten it before committing.

ASSISTANT

The first compile passed, but while reviewing the service I spotted two improvements worth making before the commit: fix the date regex and sort statuses in the lab's logical order rather than alphabetically. I'm applying those now.

ASSISTANT

The service now has explicit `listActiveTasks` and `listArchivedTasks` helpers, plus logical status sorting. I'm rerunning the checks and then I'll do a small runtime smoke test against a throwaway SQLite database.

ASSISTANT

The static checks pass. I'm checking whether this Node version can run TypeScript directly for a one-off service smoke test; if it cannot resolve the TS modules cleanly, I'll leave runtime service testing for the Vitest segment where we'll add proper test infrastructure.

ASSISTANT

Direct Node TS execution can't resolve Next-style extensionless TS imports, which is expected. I'm doing a throwaway CommonJS compile into `/tmp` instead, then I'll run the service against an in-memory database with the generated SQL schema.

ASSISTANT

The throwaway runtime smoke test passed: create, update, archive, active list, and archived list all behaved correctly against an in-memory SQLite database. I'm doing the final diff review now before committing Segment 3.

ASSISTANT

The tracked changes are scoped to the DB client and new task service. I'm staging and committing Segment 3 with the required AI attribution trailer.

ASSISTANT

The staged diff is clean: two files, no whitespace issues. I'm creating the commit now.

ASSISTANT

Segment 3 is committed. I'm checking final status and recent history so the handoff includes the exact state.

ASSISTANT

Segment 3 is complete and committed.

Commit: 772cf6f Implement task persistence operations

What changed: - Added `src/tasks/service.ts` with: - `createTask` - `updateTask` - `archiveTask` - `getTaskById` - `listTasks` - `listActiveTasks` - `listArchivedTasks` - derived `isTaskOverdue` - Updated `src/db/client.ts` so the default DB connection is lazy, which will help later throwaway DB tests.

Verified: - `npm run build` - `npm test` - `git diff --check` - Runtime smoke test against an in-memory SQLite DB: create, update, archive, active list, archived list.

Review this carefully: - `src/tasks/service.ts`: validation, sorting, archive behavior, and overdue derivation. - `src/db/client.ts`: lazy DB client setup.

What to learn from this segment: - Overdue is calculated from `dueDate < today` and `status !== "Complete"`. - Archive updates `archivedAt`; it does not delete rows. - Status sorting uses workflow order: `Todo`, `In-Progress`, `Complete`.

Test yourself:

```
npm run build
npm test
npm run db:migrate
```

Current state: local `main` is ahead of `origin/main` by Segment 3. I stopped here as planned.

USER

Next segment 4

ASSISTANT

I'll do Segment 4 only: wire the service into the homepage, add a create-task form, and show the active task list with status and overdue indication. I'll verify and commit, then stop again for your review.

ASSISTANT

I'm adding a server action plus a small client form component. The page will remain a server-rendered app screen, but it will be marked dynamic so the

production build does not try to prerender database contents.

ASSISTANT

The create action and form are added. Next I'm replacing the placeholder homepage with the actual working surface: active-task count, creation form, and active task list with overdue badges.

ASSISTANT

The page now renders real data. I'm updating the CSS for a usable app layout, form controls, task rows, and overdue/status states.

ASSISTANT

The UI styling is in place. I'm running the production build now because server actions and database-backed server components are the main risk in this segment.

ASSISTANT

Build and tests pass. I'm doing a quick local server smoke test next, mainly to confirm the dynamic homepage renders instead of only compiling.

ASSISTANT

The dev server is starting. I'm waiting for it to finish booting, then I'll request the homepage and shut it down.

ASSISTANT

The sandbox blocked binding to `127.0.0.1:3000`. I'm rerunning the dev server with local networking approval so I can smoke-test the rendered page.

ASSISTANT

Port 3000 is already in use on this machine, so I'm using port 3001 for the smoke test.

ASSISTANT

The dev server is up on `http://127.0.0.1:3001`. I'm requesting the page once from the terminal, then I'll stop the server so there's no background process left running.

ASSISTANT

The sandboxed `curl` could not reach the escalated dev server process, so I'm making the request with the same local-network permission.

ASSISTANT

The homepage rendered successfully from the dev server and shows the create form plus empty active list. I'm stopping the server now.

ASSISTANT

The server smoke test exposed two CSS compatibility warnings from `align-items: end/start`. I'm fixing those to `flex-end/flex-start` before committing.

ASSISTANT

I'm rerunning the checks after the CSS warning fix. If those pass, I'll commit Segment 4 and then start the dev server on an available port for you to try physically.

ASSISTANT

The segment checks pass. I'm reviewing the final changed files and then I'll create the commit with the required AI attribution trailer.

ASSISTANT

The review looks scoped to Segment 4. I'm staging the UI/action files and committing now.

ASSISTANT

The staged diff is clean. I'm creating the Segment 4 commit with the AI attribution trailer.

ASSISTANT

Segment 4 is committed. I'm starting a dev server on port 3001 so you can test the page physically; port 3000 was already occupied earlier.

ASSISTANT

The dev server is starting; I'm waiting until it reports ready so I can give you a reliable URL.

ASSISTANT

Segment 4 is complete and committed.

Commit: 5ca6a1f Build task creation and active task list UI

Dev server is running here:

`http://127.0.0.1:3001`

What to test physically: - Open the page. - Create a task with title, topic, due date, description, and status. - Confirm it appears under **Active Tasks**. - Create a task with a past due date and status **Todo**; confirm it shows **Overdue**. - Refresh the page; the task should still be there.

Review these files: - `app/page.tsx`: active task rendering. - `app/components/create-task-form.tsx`: form behavior. - `src/tasks/actions.ts`: server action that creates tasks.

Verified: - `npm run build` - `npm test` - Homepage rendered successfully from the dev server.

USER

`<image name=[Image #1] path="/mnt/c/Users/PC/AppData/Local/Temp/tmp4DC8.png">`
`[Image #1]`

ASSISTANT

That error is from Next's server-action rule: a `"use server"` file can export async functions, but I also exported a plain object from `src/tasks/actions.ts`. I'm moving that initial form state into the client component, then I'll rebuild and commit the fix.

ASSISTANT

The invalid export is removed from the server file. I'm running the build and tests now to confirm Next accepts the server action shape.

ASSISTANT

The build now passes, so the specific server-action error is fixed. I'm checking the running dev server and requesting the page again to confirm your browser can refresh cleanly.

ASSISTANT

The fixed page now returns HTTP content successfully from the running server. I'm committing this Segment 4 bugfix separately so the history stays honest

and reviewable.

ASSISTANT

Fixed. The issue was `src/tasks/actions.ts` exporting a plain object from a `"use server"` file.

New commit: 40c70b1 Fix task form server action exports

Verified: - `npm run build` - `npm test` - Page request to `http://127.0.0.1:3001` now returns the app HTML successfully.

Refresh your browser at:

`http://127.0.0.1:3001`

The dev server is still running.

USER

Works nicely, i assume the editing and archive functions are in the later segments?

ASSISTANT

Yes. Segment 4 only covers:

- Creating tasks
- Showing the active task list
- Showing status
- Showing overdue indication
- Persistence through SQLite

Editing and archiving are Segment 5.

Segment 5 will add: - Edit controls for existing tasks - Ability to update title, description, due date, topic, and status - Archive action - Active list excludes archived tasks - Separate archived view where archived tasks remain visible

Sorting is Segment 6, then proper deterministic tests are Segment 7.

USER

2026-07-30 Africa/Johannesburg /home/uzair/dev/uni/Software Design Project Labs/Lab 1:root/home/uzair/dev/uni/Software Design Project Labs/Lab 1:slash_tmp:tmpdir/home/uzair/dev/uni/Software Design Project Labs/Lab 1/.git/home/uzair/dev/uni/Software Design Project Labs/Lab 1/.agents/home/uzair/dev/uni/Software Design Project Labs/Lab 1/.codex

USER

Alright next segment

ASSISTANT

I'll move into Segment 5 now: editing, archiving, and an archived-task view. I'll first stop/check any leftover dev server state and confirm the repo is clean, then make the changes and commit only this segment.

ASSISTANT

The repo is clean and the last pushed state already includes the Segment 4 bugfix. I'm reading the action and style files next so the edit/archive work reuses the existing server-action pattern instead of introducing a second workflow.

ASSISTANT

There may still be a dev server from the previous segment, so I'm stopping that now before changing the page. After the commit I can start a fresh one for your Segment 5 review.

ASSISTANT

The previous dev-server session is already gone, so there's nothing to clean up. I'm implementing the edit/archive actions and a reusable task list component now.

ASSISTANT

The edit UI will be inline per active task so the marker can update one task at a time. The archived section will be read-only for now, because the lab only requires archived tasks to remain viewable, not restorable.

ASSISTANT

The server actions now cover update and archive using the same validation path as creation. I'm adding the client-side task card component next, with an edit form and archive action on each active task.

ASSISTANT

The task card component is in place. I'm updating the homepage to use it and to render the separate archived list, so archived tasks remain visible without mixing into the active list.

ASSISTANT

The page now uses the reusable cards and includes an archived section. I'm expanding the CSS for inline edit controls, archive buttons, and the wider three-counter summary.

ASSISTANT

The UI changes are in place. I'm running the production build, tests, and whitespace checks now; server actions are again the main thing to verify.

ASSISTANT

The build and test commands pass. I'm starting the dev server on port 3001 again and checking that the page renders after the edit/archive changes.

ASSISTANT

The server is ready. I'm requesting the page once and then I'll keep the server running for your manual review unless the request exposes an error.

ASSISTANT

The page renders with the edit forms and archive buttons, using your existing local tasks from the ignored SQLite file. I'm checking the server log for runtime errors, then I'll commit the Segment 5 changes.

ASSISTANT

No runtime errors showed in the dev server log. I'm reviewing the final diff and then committing Segment 5 with the AI attribution trailer.

ASSISTANT

The diff is scoped, but I spotted a couple of long JSX lines in the new card component. I'm wrapping those for readability before the final commit.

ASSISTANT

The cleanup is done. I'm rerunning the verification commands once more after the final formatting pass, then I'll commit.

ASSISTANT

The final checks pass. I'm making one last page request against the running dev server, then I'll commit the Segment 5 work.

ASSISTANT

The final page request succeeded with active tasks, edit forms, archive buttons, and the archived section present. I'm staging and committing now.

ASSISTANT

The staged changes are clean: one new task-card component plus page/action/style updates. I'm creating the Segment 5 commit with the required AI trailer.

ASSISTANT

Segment 5 is committed. I'm checking final status and the dev server state so I can leave you with exactly what to test.

ASSISTANT

Segment 5 is complete and committed.

Commit: d6fb61f Add task editing and archive workflow

Dev server is running here:

<http://127.0.0.1:3001>

What changed: - Added inline editing for active tasks. - Added archive buttons for active tasks. - Added a separate **Archived Tasks** section. - Active list now excludes archived tasks; archived tasks remain visible. - Added an archived counter in the page summary.

Review: - app/components/task-card.tsx: edit/archive UI. - src/tasks/actions.ts: update/archive server actions. - app/page.tsx: active vs archived rendering.

Test physically: - Edit a task title/topic/date/status and save. - Refresh the page and confirm the edit persists. - Archive a task and confirm it leaves **Active Tasks**. - Confirm the same task appears under **Archived Tasks**. - Confirm the archived counter updates.

Verified: - `npm run build` - `npm test` - `git diff --check` - Page rendered successfully on the dev server.

USER

Did the pdf mention having a status feature?

ASSISTANT

Yes. `lab_one.pdf` explicitly requires task status.

It says each task has one of three fixed statuses:

Todo, In-Progress, Complete

Important details: - These statuses are **not user-customisable**. - Overdue must **not** be a status. - Overdue should be indicated separately, derived from the due date.

So the current status dropdown is required and correct.

USER

alright next segment

USER

The user interrupted the previous turn on purpose. Any running unified exec processes may still be running in the background. If any tools/commands were aborted, they may have partially executed.

USER

do the next segment

ASSISTANT

I'll do Segment 6 only: add controls for sorting by topic, status, and due date, wire them into the existing service, verify, commit, then stop for your review. I'll check the repo and current server state first so this stays scoped.

ASSISTANT

The current code is clean and already pushed through Segment 5. I'm adding query-string based sorting now, so the active list can be sorted by due date, topic, or fixed status order without client-only state.

ASSISTANT

The sort controls are added as links backed by `?sort=dueDate|topic|status`. I'm running build/tests now to verify the async page props and server rendering are valid.

ASSISTANT

The build and tests pass. I'm smoke-testing the three sort URLs against the running dev server now, to confirm each route renders and the active sort marker appears.

ASSISTANT

The three sort URLs rendered and each one marked the selected sort control. I'm checking the dev-server log for hidden runtime errors, then I'll review and commit the segment.

ASSISTANT

No dev-server errors showed. I'm reviewing the changed files and then I'll make the Segment 6 commit with the required AI trailer.

ASSISTANT

The diff is limited to sort controls, page query handling, and CSS. I'm staging and committing Segment 6 now.

ASSISTANT

Segment 6 is committed. I'm checking final status and recent history so the handoff is exact.

ASSISTANT

Segment 6 is complete and committed.

Commit: 8a867ce Add task sorting controls

Dev server is still running here:

`http://127.0.0.1:3001`

What changed: - Added sort controls for active tasks: - Due date - Topic - Status - Sorting uses query params: `?sort=dueDate, ?sort=topic, ?sort=status`. - Status sorting uses the service's fixed workflow order: **Todo**, **In-Progress**, **Complete**. - Archived tasks stay separate and are not affected by the active-task sort controls.

Review: - `app/components/sort-controls.tsx`: sort UI links. - `app/page.tsx`: query param parsing and sort wiring. - `app/globals.css`: sort control styling.

Test physically: - Create a few active tasks with different topics, statuses, and due dates. - Click **Due date**, **Topic**, and **Status**. - Confirm the selected control is highlighted. - Confirm archived tasks remain visible only in **Archived Tasks**.

Verified: - `npm run build - npm test - git diff --check` - Rendered all three sort URLs on the dev server.

USER

<image name=[Image #1] path="/mnt/c/Users/PC/AppData/Local/Temp/tmp1B3A.png">

[Image #1] The formatting of the page went bad

ASSISTANT

That screenshot means the CSS is not being applied at all, not just a layout regression. I'll check the running dev server logs and the generated CSS route first, then fix the cause and commit it separately if code changes are needed.

ASSISTANT

The server is returning the page without logging an obvious CSS compile error. I'm checking the CSS asset response directly now, because if that route is 404 or returning the wrong content the browser will render exactly like your screenshot.

ASSISTANT

The CSS file the page is linking is returning 404, which explains the raw HTML look. This is likely because I ran `next build` while the dev server was still running, so the `.next` output got replaced under the dev server. I'm restarting the dev server cleanly.

ASSISTANT

The stale dev server is stopped. I'm starting a fresh one on port 3001 and then I'll verify both the page and stylesheet are served correctly.

ASSISTANT

The server is fresh and ready. I'm checking the page and extracting the actual CSS link, then I'll verify that stylesheet returns 200.

ASSISTANT

Fixed. It was a dev-server state issue, not a code styling issue.

Cause: I had run `npm run build` while the dev server was still running. That rewrote `.next/`, so the running dev server started linking to a CSS file that returned 404. Browser then rendered plain unstyled HTML.

I restarted the dev server cleanly, and the stylesheet now returns 200 OK.

Use:

`http://127.0.0.1:3001`

Do a hard refresh in the browser: - Windows/Linux: `Ctrl + Shift + R` - Or clear the tab cache/reopen the page

No new commit was needed because the source files were already correct.

USER

Next segment

ASSISTANT

I'll do Segment 7 now: add deterministic Vitest coverage for the task service using throwaway in-memory SQLite, verify `npm test` and build, commit, then stop for review. I'll keep this focused on tests rather than changing app behavior.

ASSISTANT

The tree is clean and Segment 6 is already pushed. I'm adding the test file now; it will create a fresh in-memory SQLite database per test and apply the generated migration SQL, so tests do not depend on your local `data/todo.db`.

ASSISTANT

The test coverage is in place: create, edit, archive, overdue derivation, and sorting. I'm running `npm test` now and expecting it to catch any mismatch in the service's validation or sorting behavior.

ASSISTANT

All five service tests pass. I'm running the production build as the second required check, then I'll review the diff for any cleanup before committing.

ASSISTANT

The build still passes after adding tests. I'm reviewing the test file and diff now, mainly to make sure the tests are deterministic and not tied to your local database.

ASSISTANT

The tests are using the in-memory DB correctly. I'm doing a small readability cleanup on the long assertions before staging, without changing test behavior.

ASSISTANT

I ran a build while the dev server was still running, which can stale the dev CSS in this Next setup. I'll finish the test commit first, then restart the dev server cleanly so the browser view stays styled.

ASSISTANT

The cleaned tests and build both pass. I'm staging and committing the test segment now.

ASSISTANT

Only the new test file is untracked, as expected for this segment. I'm adding it and committing with the AI attribution trailer.

ASSISTANT

The test file is staged. I'm checking the staged diff once, then creating the commit.

ASSISTANT

Segment 7 is committed. I'm restarting the dev server cleanly now because the production build can invalidate the dev CSS cache while the server is running.

ASSISTANT

The old server is stopped. I'm starting a fresh one on 3001 and verifying the stylesheet returns 200 before I hand this back.

USER

The user interrupted the previous turn on purpose. Any running unified exec processes may still be running in the background. If any tools/commands were aborted, they may have partially executed.

USER

Is there anything i need to test for segment 7?

ASSISTANT

I'll quickly check whether the dev server finished starting after the interruption, then I'll give you the exact Segment 7 test checklist.

ASSISTANT

The dev-server command is still running but has not printed readiness output, so I'm stopping that background process; Segment 7 only needs command-line tests, not browser testing.

ASSISTANT

For Segment 7, the main thing to test is the automated test command:

```
npm test
```

Expected result: 5 tests pass in `src/tasks/service.test.ts`.

Also run:

```
npm run build
```

What Segment 7 proves: - Creating a task stores all required fields. - Editing persists changed fields. - Archiving removes the task from active tasks but keeps it in archived tasks. - Overdue is derived from due date/status, not stored as a status. - Sorting works by topic, status, and due date. - Tests use an in-memory throwaway SQLite DB, not your real `data/todo.db`.

No browser testing is required for Segment 7, because it was only the test coverage segment.

USER

so where is the data getting saved to exactly

ASSISTANT

Your real app data is saved here:

```
SDP-Lab-1/data/todo.db
```

Full path:

```
/home/uzair/dev/uni/Software Design Project Labs/Lab 1/SDP-Lab-1/data/todo.db
```

That file is a local SQLite database. It is intentionally ignored by git, so your personal tasks are not committed.

The tests do **not** use that file. Segment 7 tests use an in-memory SQLite database:

```
:memory:
```

So running `npm test` will not affect your real task data.

USER

Do the next segment

ASSISTANT

I'll do Segment 8 now: documentation and final compliance pass. I'll update the markdown docs with the required rubric sections, AI declarations, exact run/test commands, and schema explanation, then verify and commit.

ASSISTANT

The repo is clean and already has nine commits, so the commit-history requirement is satisfied. I'm collecting exact version/schema details now so the README matches the shipped app rather than approximate notes.

ASSISTANT

I have the exact Node/npm versions and schema. I'm replacing the placeholder README with the required documentation sections and AI declarations, plus a transcript note for submission.

ASSISTANT

The README now contains the required sections: Third-Party Code, Database Design, Running It, Testing, and AI Usage Declaration. I'm reviewing it and then I'll run the final verification commands.

ASSISTANT

The README passes whitespace checks. I'm adding the repository link and then I'll run the documented commands that matter for final marking: migration, tests, and build.

ASSISTANT

The README now includes the GitHub repository URL. I'm running `npm run db:migrate`, `npm test`, and `npm run build` to verify the documented workflow still works.

ASSISTANT

The documented migration, test, and build commands all pass. I'm doing a final compliance check of the README content and commit history, then I'll commit this documentation segment.

ASSISTANT

The README contains all required sections, the commit count is 9 before this documentation commit, and the diff has no whitespace issues. I'm committing Segment 8 now.

ASSISTANT

The README is staged. I'm checking the staged diff once and then creating the documentation commit with the required AI trailer.

ASSISTANT

Segment 8 is committed. I'm checking final status, commit count, and recent history so the closeout is precise.

ASSISTANT

Segment 8 is complete and committed.

Commit: `3ecc696 Document setup database design and AI usage`

What changed: - Replaced the placeholder README.md with the required documentation: - `Running It` - `Third-Party Code` - `Database Design` - `Testing` - `AI Usage Declaration` - Added exact Node/npm versions and commands. - Documented the SQLite schema, archive design, overdue derivation, and test behavior. - Added the GitHub repo URL and AI policy declarations.

Verified: - `npm run db:migrate` - `npm test` passes: 5 tests - `npm run build` - `git diff --check`

Repo status: - Working tree is clean. - Commit count is now 10, so the “at least 6 commits” requirement is satisfied.

Review: - README.md

For submission, you still need to include/export the AI usage transcript from this Codex conversation alongside the GitHub repository link.

USER

what is “AI in-line editing tools”

ASSISTANT

“AI in-line editing tools” means AI assistance built directly into an editor while you type, usually as autocomplete or small inline rewrites.

Examples: - GitHub Copilot autocomplete in VS Code - Cursor inline edits - VS Code AI autocomplete - Tabnine - Codeium/Windsurf autocomplete - JetBrains AI inline suggestions

It is different from **AI code generation** through Codex/ChatGPT where you ask for a chunk of work and apply/review the result.

In our README, I wrote:

`This repository does not use AI in-line editing tools.`

That means we are declaring that the project did not use editor-based autocomplete/inline AI suggestions. If you personally used something like Copilot while editing, we should change that line to declare it.

USER

I want you to now review through the entire project in line with the rubric and check in detail if passes all requirements for the highest marks

ASSISTANT

I'll do this as a rubric-focused review, not an implementation segment: inspect the actual repo, run the key commands, and report any gaps that could cost marks. I'll avoid changing files unless you explicitly ask for fixes after the review.

ASSISTANT

I'm going through this as a marking review now: rubric compliance, AI policy compliance, functional behavior, tests, database design, and commit evidence. I'll separate "will lose marks" from "looks acceptable" in the final report.

ASSISTANT

The first concrete risk I found is AI-policy related, not app behavior: the newest documentation commit lacks the **Assisted-by: Codex[GPT-5]** trailer, while the earlier Codex-assisted commits have it. I'm checking the source and tests next to see whether there are functional or rubric issues beyond that.

ASSISTANT

Source review so far: the core fields, status enum, archive model, and overdue derivation all line up with the brief. I'm running the actual verification commands now so the final assessment can distinguish "readable code looks right" from "verified on this machine."

ASSISTANT

The project passes `npm test` and `npm run build` in the working tree. I'm doing one clean-clone smoke test as well, because the rubric's first step is "can the marker clone and run it from scratch?"

ASSISTANT

The clean clone was created successfully in `/tmp`, so the repository itself is self-contained enough to clone locally. I'm installing from the lockfile in that clone and then running the documented `migrate/test/build` path.

ASSISTANT

The clean-clone install failed inside the sandbox while `better-sqlite3` was rebuilding and npm tried to write under the home npm cache/log directory.

That looks sandbox-related, so I'm rerunning the same install with approval rather than treating it as a project failure.

ASSISTANT

The approved clean install is still running; native SQLite packages often take a little longer because they compile a binding. I'll wait for it before running the documented commands in the clone.

ASSISTANT

Clean-clone verification passed the documented migrate/test/build path. One non-rubric issue appeared during install: `npm audit` reports vulnerabilities, so I'm checking whether those are production dependencies or only tooling before I classify the risk.

ASSISTANT

The audit is a real dependency warning, but it is outside the explicit lab functionality marks. I'm re-checking the PDFs briefly so I don't rely only on memory for the marking categories.

ASSISTANT

`pdftotext` is not installed here, but the rubric points I extracted earlier are enough for this review: local SQLite app behavior, docs, database design, testing, commit evidence, and AI-policy declarations/transcripts. I'm checking for any hidden generated files or data files before finalizing.

ASSISTANT

Repository hygiene looks good: generated build output, `node_modules`, and the local database are not tracked; only `data/.gitkeep` is tracked so the folder exists. I'm now ready to give the mark-focused review.

ASSISTANT

Findings

1. **AI-policy commit trailer gap** latest commit `2b993e3` changed AI documentation but does **not** include the required **Assisted-by: Codex[GPT-5]** trailer. Earlier Codex-assisted commits do include it. If that latest commit was AI-assisted, this is a policy risk and should be fixed before final submission.
2. **AI transcript submission still needs handling** `README.md` declares AI use clearly, but the repo currently does not include/export the

conversation transcripts. If the PDF requires transcripts as part of submission, you must submit them alongside the repo link. This is probably the biggest remaining mark risk for the AI-policy section.

3. **Dependency audit warning** `npm audit --omit=dev` reports 3 high-severity production dependency advisories through `next` transitive dependencies: `postcss` and `sharp`. This is not part of the core lab rubric, but a strict marker may notice. The project still builds and runs, and npm’s suggested fix is breaking/unhelpful, so I would treat this as a documented residual risk rather than changing framework versions blindly.

Rubric Check

The app itself looks strong against the lab requirements.

The required task fields are implemented: title, description, due date, topic, and status in `src/db/schema.ts` and the forms in `create-task-form.tsx` / `task-card.tsx`.

Status values are exactly **Todo**, **In-Progress**, and **Complete**, enforced in both code and DB constraints: `schema.ts` and migration SQL.

Archive is implemented without deleting data: `service.ts`. Archived tasks remain visible in the UI: `page.tsx`.

Overdue is derived correctly from due date and status, not stored as a fake status: `service.ts`.

Database design is appropriate for this lab: local SQLite at `data/todo.db`, one `tasks` table, useful constraints, and indexes. The README documents this clearly in `README.md`.

Testing looks sufficient for high marks: `service.test.ts` has 5 deterministic tests covering create, edit, archive, overdue, and sorting. The tests use in-memory SQLite and the shipped migration.

Documentation covers the required sections: **Running It**, **Third-Party Code**, **Database Design**, **Testing**, and **AI Usage Declaration** in `README.md`.

Commit history satisfies the “at least 6 commits” requirement: there are 11 commits total, spread across July 29 and July 30. The work is segmented clearly.

Verification Run

I verified:

```
npm test
npm run build
```

Both passed in the working repo.

I also tested a clean local clone with:

```
npm ci
npm run db:migrate
npm test
npm run build
```

Those passed too, after allowing `npm ci` to write npm cache/native build files.

Overall Assessment

Functionality, database design, README documentation, tests, and commit count are all in good shape for high marks. The two things I would fix before submission are the missing **Assisted-by: Codex[GPT-5]** trailer on the latest documentation commit and making sure the AI transcripts are submitted/exported as required by the policy.

USER

<image name=[Image #1] path="/mnt/c/Users/PC/AppData/Local/Temp/tmp181A.png">
[Image #1] What's the thing at the bottom left btw?

ASSISTANT

That bottom-left circle is the **Next.js development indicator**.

It appears only when you run the app with `npm run dev`. It gives quick access to Next.js dev tools, route info, preferences, and Turbopack options.

It will **not appear in production** when the marker runs:

```
npm run build
npm start
```

So it is normal and not part of your app UI.

USER

For the rubric, what about this “and at least one clear instance of the author identifying an unsuitable or incorrect output and redirecting it”

ASSISTANT

That means the transcript must show **you catching a problem in the AI's output and steering it back**.

For your project, you already have good examples:

1. **Server action error** You sent the screenshot with:

A "use server" file can only export async functions,
found object.

That shows you identified that the generated implementation was incorrect. We then fixed the server action export issue.

2. **Broken page formatting** You sent the screenshot saying:

“The formatting of the page went bad”

That shows you reviewed the AI-generated UI physically, found an unsuitable result, and redirected the work toward fixing it.

Those are exactly the kind of transcript evidence the rubric is asking for. In your submission, make sure the exported transcript includes those moments. They prove you were not just accepting AI output blindly.

USER

Remove “COMS3011A Lab 1” from the page, unless the pdf said it must be there?

ASSISTANT

The rubric requirement is about documentation/submission identity, not showing COMS3011A Lab 1 in the UI. I’m going to remove that visible eyebrow text from the page and clean up the now-unused CSS.

ASSISTANT

I’m only removing the visible page label, leaving the README and browser metadata intact because those help identify the lab submission and don’t affect the app screen.

ASSISTANT

The UI text is removed now. I’m running the build to make sure the page still type-checks, then I’ll commit this with the AI trailer so it stays policy-compliant.

ASSISTANT

Build passed after the change. I’m committing just these two UI files with the required **Assisted-by** trailer.

ASSISTANT

Removed the visible COMS3011A Lab 1 label from the page header. The PDF did not require it to be shown in the UI.

I left the README title and browser metadata alone because those are useful for submission identity. Verified with:

```
npm run build
```

Committed as:

```
192bbe2 Remove lab label from app header
```

```
Assisted-by: Codex[GPT-5]
```

USER

I want you to include this in the markdown: “Student Number: 2802191, Name: Uzair Mahomed” so that I have identification on there. Also can you add a slider somewhere to switch between light mode and dark mode? And make sure that when the person refreshes the page, the appearance mode is remembered

ASSISTANT

I’ll add the student identity to the README only, then implement a persisted light/dark toggle in the app header. Since the preference must survive refreshes, I’ll store it in `localStorage` and apply it to the document theme.

ASSISTANT

I’m adding this as a small client component instead of mixing browser-only `localStorage` calls into the server page. The page will render normally on the server, and the toggle will take over once the browser loads.

ASSISTANT

The component and README update are in place. I’m updating the CSS variables and switch styling now so dark mode changes the whole interface, including panels, inputs, task cards, and status badges.

ASSISTANT

The dark theme now uses the same CSS variable system as the existing design, so it should switch consistently rather than patching individual elements one by one. I’m running tests and a production build next.

ASSISTANT

Tests and build both pass. I’m committing the README identification and persisted appearance toggle together as one focused change with the required AI-policy trailer.

ASSISTANT

The diff is limited to the README, layout, page header, CSS theme variables, and the new toggle component. I’ll stage those exact files and commit.

ASSISTANT

The commit was created. I'm doing one final status check so I can tell you exactly what changed and whether anything is left uncommitted.

ASSISTANT

Done.

I added the identification line to README.md:

`Student Number: 2802191, Name: Uzair Mahomed`

I also added a light/dark mode slider in the page header via `theme-toggle.tsx`. It saves the choice in `localStorage`, so refreshing the page keeps the selected appearance mode.

Verified:

```
npm test
npm run build
```

Committed as:

`9809971 Add student details and theme toggle`

`Assisted-by: Codex[GPT-5]`

Your local branch is now ahead of `origin/main` by 2 commits, so you still need to push.

USER

Change “Create tasks with a topic and due date, then track their current status from one local SQLite database.” to “Create tasks with a topic and due date, then track their current status” so it sounds better

ASSISTANT

I'll update that one sentence in the page header, then run a quick build check and commit the wording change with the required trailer.

ASSISTANT

The copy change is in place. I'm running the build to make sure this tiny edit didn't disturb the app, then I'll commit it.

ASSISTANT

Build passed. I'm committing just the wording change now.

ASSISTANT

Updated the header sentence to:

Create tasks with a topic and due date, then track their current status.

Verified with:

```
npm run build
```

Committed as:

```
f6ffd1e Refine app header copy
```

```
Assisted-by: Codex[GPT-5]
```

USER

Where can i get the transcript of this chat?

ASSISTANT

Usually in the Codex/ChatGPT UI there's a **share/export** option for the conversation.

Look for one of these:

- **Share** button near the top of the chat.
- **Export / Download** option in the chat menu.
- Browser menu or app menu option like **Export conversation**.
- If no export exists, use **Print** → **Save as PDF** from the browser.

For the rubric, make sure the saved transcript includes:

- The planning segment.
- Your AI-policy/rubric instructions.
- The server-action error screenshot and fix.
- The formatting issue screenshot and fix.
- The testing/build verification.
- The final review against the rubric.

That should satisfy the “author identified unsuitable/incorrect AI output and redirected it” requirement.

USER

im in the terminal version of codex

ASSISTANT

I'll check the local Codex files first, because terminal Codex usually keeps session state/logs somewhere under its config directory. Then I'll tell you the safest way to export something suitable for the rubric.

ASSISTANT

I'm using the OpenAI/Codex docs skill for this because it's specifically about the terminal Codex product. I'll use local/manual info first and avoid guessing paths that may differ by install.

ASSISTANT

The official manual fetch failed because this sandbox cannot resolve `developers.openai.com` right now. I'm falling back to checking your local Codex directory for recorded session files.

ASSISTANT

Your CLI does have a `~/.codex/sessions` directory, which is likely the relevant transcript store. I'm locating the current/recent session file and checking its format without exposing unrelated private history.

ASSISTANT

The current thread is indexed as `SDP-Lab-1-Codex-Assisted`, and the active transcript file is a JSONL file under `~/.codex/sessions`. I'm checking a few records so I can give you a usable export command, not just a raw path.

ASSISTANT

In terminal Codex, the raw transcript is stored as a JSONL session file.

For this chat, the file is:

```
/home/uzair/.codex/sessions/2026/07/29/rollout-2026-07-29T19-33-17-019faef0-1672-7e21-92ee-7
```

You can inspect it with:

```
less ~/.codex/sessions/2026/07/29/rollout-2026-07-29T19-33-17-019faef0-1672-7e21-92ee-750aa7
```

There is also:

```
/home/uzair/.codex/history.jsonl
```

but that only stores your prompts, not the full back-and-forth. For the rubric, the `sessions/...jsonl` file is the important one.

Before submitting it, skim it because it may include tool outputs, local paths, and session metadata. The key evidence the marker wants is in that session file: planning, your corrections/screenshots, testing, commits, and AI-policy discussion.

USER

Do i include it in the repo or just remember to submit it individually when submissions open up?

ASSISTANT

Submit it **individually when submissions open**, unless the PDF explicitly says transcripts must be committed inside the repository.

I would **not put the raw JSONL directly into the repo** because it can include local paths, tool outputs, metadata, and possibly unrelated context. Better approach:

1. Keep the repo clean.
2. Export or convert the relevant transcript later.
3. Submit it as a separate file with your repo link.

A sensible filename would be:

2802191_Uzair_Mahomed_AI_Transcript.jsonl

or, if you convert it to PDF/text:

2802191_Uzair_Mahomed_AI_Transcript.pdf

Before uploading, quickly skim it to make sure it includes the moments where you corrected the AI output.

USER

How do you convert it to a pdf?

ASSISTANT

Best path: convert the raw Codex JSONL transcript into a readable Markdown file, then convert that Markdown to PDF.

Use this from your terminal:

```
SESSION="$HOME/.codex/sessions/2026/07/29/rollout-2026-07-29T19-33-17-019faef0-1672-7e21-92a
```

```
jq -r '
  select(.type == "response_item" and .payload.type == "message")
  | "## " + (.payload.role | ascii_upcase) + "\n"
  + ([.payload.content[]? | .text?] | join("\n\n"))
  + "\n"
' "$SESSION" > "$HOME/2802191_Uzair_Mahomed_AI_Transcript.md"
```

Then convert to PDF with **pandoc**:

```
pandoc "$HOME/2802191_Uzair_Mahomed_AI_Transcript.md" \
-o "$HOME/2802191_Uzair_Mahomed_AI_Transcript.pdf"
```

If `pandoc` is not installed:

```
sudo apt install pandoc texlive-xetex
```

The PDF will be here:

```
~/2802191_Uzair_Mahomed_AI_Transcript.pdf
```

Also keep the original JSONL file just in case the marker wants the raw transcript.

USER

Would it keep the session file untouched? i dont want to get rid of it by accident

ASSISTANT

Yes. That command **does not modify or delete** the session file.

This part only reads it:

```
"$SESSION"
```

This part writes a new Markdown file somewhere else:

```
> "$HOME/2802191_Uzair_Mahomed_AI_Transcript.md"
```

So the original stays untouched at:

```
~/codex/sessions/2026/07/29/rollout-2026-07-29T19-33-17-019faef0-1672-7e21-92ee-750aa7baa71
```

To be extra safe, you can make a backup first:

```
cp "$HOME/.codex/sessions/2026/07/29/rollout-2026-07-29T19-33-17-019faef0-1672-7e21-92ee-750aa7baa71"
"$HOME/2802191_Uzair_Mahomed_AI_Transcript_raw.jsonl"
```

That also leaves the original untouched.

USER

```
uzair@DESKTOP-PUCN3E5:~/dev/uni/Software Design Project Labs/Lab 1/SDP-Lab-1$ pandoc "$HOME/2802191_Uzair_Mahomed_AI_Transcript.md"
-o "$HOME/2802191_Uzair_Mahomed_AI_Transcript.pdf" Error producing
PDF. ! You can't use 'macro parameter character #' in math mode. 1.328
...kills/.system/skill-installer/SKILL.md) #
```

ASSISTANT

Yes, that error is from Pandoc/LaTeX interpreting part of the transcript as math because the transcript contains things like `$HOME`, `$CODEX_HOME`, and `#`.

Run this instead:

```
pandoc "$HOME/2802191_Uzair_Mahomed_AI_Transcript.md" \
  --from markdown-tex_math_dollars-tex_math_single_backslash \
  --pdf-engine=xelatex \
  -o "$HOME/2802191_Uzair_Mahomed_AI_Transcript.pdf"
```

That tells Pandoc not to treat $\dots$ as math.

If it still fails, make a plain HTML file and print it to PDF from your browser:

```
pandoc "$HOME/2802191_Uzair_Mahomed_AI_Transcript.md" \
  --from markdown-tex_math_dollars-tex_math_single_backslash \
  -o "$HOME/2802191_Uzair_Mahomed_AI_Transcript.html"
```

Then open the HTML file and use **Print** → **Save as PDF**.

None of these commands touch the original Codex session file. They only read the Markdown copy and write a new PDF/HTML file.

USER

So do you think i should give it as a pdf or does it not matter?

ASSISTANT

I'd submit it as **PDF** if the submission portal accepts PDFs.

Reason: PDF is easier for the marker to read, preserves formatting better than raw JSONL, and looks more intentional. Keep the raw JSONL as a backup, but don't submit the raw file unless they specifically ask for it.

Best submission set:

GitHub repo link
2802191_Uzair_Mahomed_AI_Transcript.pdf

If the PDF is too large or the portal rejects it, submit the Markdown version instead:

2802191_Uzair_Mahomed_AI_Transcript.md

USER

2026-08-02 Africa/Johannesburg /home/uzair/dev/uni/Software Design
Project Labs/Lab 1:root/home/uzair/dev/uni/Software Design Project
Labs/Lab 1:slash_tmp:tmpdir/home/uzair/dev/uni/Software Design Project
Labs/Lab 1/.git/home/uzair/dev/uni/Software Design Project Labs/Lab
1/.agents/home/uzair/dev/uni/Software Design Project Labs/Lab 1/.codex

USER

Im trying to open it with terminal but its not working uzair@DESKTOP-PUCN3E5:/mnt/c/Users/PC/Desktop/Software Design Project Labs/Lab 1/SDP-Lab-1\$ npm run dev

```
sdp-lab-1@0.1.0 dev next dev
```

ASSISTANT

I'll check whether the dev server is actually running, then compare the repo path you're using under `/mnt/c` with the project path I've been editing under `/home/uzair`. That path difference can matter in WSL.

ASSISTANT

There are two copies/paths involved: the project I've been editing is under `/home/uzair/dev/...`, while your terminal is running the copy under `/mnt/c/Users/PC/Desktop/...`. I'm checking whether that Windows-mounted copy has the latest commits and dependencies.

ASSISTANT

The `/mnt/c` copy is up to date, so this is not a stale-copy problem. I'm going to start the dev command in that same path and watch the output for the point where it gets stuck.

ASSISTANT

I reproduced it: `next dev` exits immediately after printing the script line, without starting a server. That usually means the local `node_modules/.bin/next` shim is wrong for this environment, so I'm checking the installed binary and package state.

ASSISTANT

The problem is your local `next` command is broken in that `/mnt/c/...` copy.

I checked it and found this:

```
node_modules/.bin/next: empty
```

So when you run:

```
npm run dev
```

it prints:

```
> next dev
```

then exits, because the `next` executable shim is an empty file.

Fix it by reinstalling dependencies:

```
cd "/mnt/c/Users/PC/Desktop/Software Design Project Labs/Lab 1/SDP-Lab-1"

rm -rf node_modules .next
npm ci
npm run db:migrate
npm run dev
```

Then open:

`http://localhost:3000`

If it still gives port issues, run:

```
npm run dev -- --hostname 127.0.0.1 --port 3001
```

and open:

`http://localhost:3001`

Better long-term: run the project from the WSL Linux path instead of `/mnt/c`, because Node projects are usually more reliable there:

```
cd "$HOME/dev/uni/Software Design Project Labs/Lab 1/SDP-Lab-1"
npm run dev
```

USER

should the read me instructions to installing this be updated with that or?

ASSISTANT

Yes, it is worth adding as a short troubleshooting note, but I would not make the broken-shim fix part of the main clean-clone instructions. The main path stays simple; the WSL/`node_modules` repair note goes under **Running It** for anyone who hits the same terminal issue.

ASSISTANT

I'm updating the README now with a compact note: prefer the WSL/Linux project path, and reinstall dependencies if the local Next executable is broken. That documents the fix without making the normal install steps look more complicated than they are.

ASSISTANT

The README note is in place. This is docs-only, so I'm checking the diff and committing it with the AI trailer; no build run is needed for this wording change.

ASSISTANT

Yes, and I added it to the README as a WSL troubleshooting note under Running It.

It now explains:

- Prefer running from the WSL Linux filesystem instead of `/mnt/c/...`
- If `npm run dev` prints `next dev` then exits, reinstall dependencies with:

```
rm -rf node_modules .next
npm ci
npm run db:migrate
npm run dev
```

Committed as:

```
d59aa25 Document WSL dev server troubleshooting
Assisted-by: Codex[GPT-5]
```

USER

Alright im back to the ubuntu path

ASSISTANT

Good. From the Ubuntu path, use:

```
cd "$HOME/dev/uni/Software Design Project Labs/Lab 1/SDP-Lab-1"
npm run dev
```

Then open:

```
http://localhost:3000
```

If port 3000 is busy:

```
npm run dev -- --port 3001
```

and open:

```
http://localhost:3001
```

USER

I'd like to change the frontend so it looks more appealing and unique, can we go about changing this into a maybe a futuristic theme style frontend. Also lets have a button that you click to "View Archieved Tasks" so its more organized

ASSISTANT

I'll treat this as a frontend refinement pass: keep the existing task behavior, change the visual system toward a futuristic dashboard style, and move archived tasks behind a dedicated toggle button. I'll inspect the current components first, then make the smallest structural change needed for the archive section.

ASSISTANT

I found two unrelated working-tree items already present: `README.md` is modified and `README.md:Zone.Identifier` exists. I'll leave both alone and only stage the frontend files I change.

ASSISTANT

The README change is the WSL note being removed, so I'll treat that as your change and not touch it. Next I'm adding a small `ArchiveSection` client component and restyling the existing CSS around a sharper, neon-accented dashboard look.

ASSISTANT

The archive list now has a real button-controlled reveal. I'm applying the visual redesign next: new color tokens for light/dark modes, a subtle grid-like app background, brighter status treatments, and crisper panels/buttons.

ASSISTANT

The main futuristic styling is in place. I'm adding the archive button layout polish now so the archived section stays compact until the user chooses to open it.

ASSISTANT

The archive panel is now collapsed by default with a single `View Archived Tasks` button. I'm running the automated tests and production build next to catch any client/server component or TypeScript issues.

ASSISTANT

Tests and build passed, so the React boundary and types are fine. I'm starting a local dev server now so you can physically inspect the redesigned page.

ASSISTANT

The dev server is running at `http://127.0.0.1:3001`. I'm doing a quick HTTP check and then I'll commit only the frontend files, leaving your

README/untracked Windows metadata alone.

ASSISTANT

The HTTP check could not connect, so I'm checking whether the dev server process died after its first request. If it did, I'll report that separately from the code change rather than hiding it.

ASSISTANT

The server is still alive and responding; the sandboxed `curl` check was misleading, but the Next logs show successful `GET / 200` responses. I'm committing the frontend-only files now.

ASSISTANT

Implemented the frontend refresh and committed it.

What changed: - Restyled the app into a sharper futuristic dashboard look with grid-style background, brighter accent colors, stronger panels, updated buttons, and improved task cards. - Added `archive-section.tsx` with a **View Archived Tasks / Hide Archived Tasks** button. - Archived tasks are now hidden by default, so the main page is more organized.

Verified:

```
npm test
npm run build
```

The dev server is running here for you to inspect:

`http://127.0.0.1:3001`

Committed as:

00a0354 Refresh task dashboard styling
Assisted-by: Codex[GPT-5]

Note: I left your existing `README.md` modification and `README.md:Zone.Identifier` untracked file untouched.

USER

You can do whatever with the `README.md`, must of been a mistake from me? Can you make it default to dark mode, i think thats like the best version of it

ASSISTANT

I'll make dark mode the default while keeping the existing "remember my choice" behavior. I'll also clean up the accidental `README` change by restoring the

WSL note, and I'll inspect the `README.md:Zone.Identifier` file before deciding whether it is safe to remove.

ASSISTANT

The README diff is just the WSL note being removed, so I'll restore that. The `Zone.Identifier` file is Windows metadata, not project content, so I'll remove it from the working tree cleanup.

ASSISTANT

I'm making the default logic dark-only when there is no saved preference. If someone switches to light mode, that saved choice still wins on refresh.

ASSISTANT

The default is now dark mode, the README note is restored, and the Windows metadata file has been removed. I'm running tests and build before committing these small follow-up fixes.

ASSISTANT

The verification passed, and the README is clean now. I'm committing the dark-mode default change only.

ASSISTANT

The commit is done. I'm checking the final working tree and the running dev server status so I can give you an accurate close-out.

ASSISTANT

Done. The app now defaults to dark mode when there is no saved preference, but it still remembers if someone switches back to light mode.

I also restored the README to a clean state and removed the accidental `README.md:Zone.Identifier` metadata file. Working tree is clean.

Verified:

```
npm test
npm run build
```

Committed as:

```
44ef128 Default app appearance to dark mode
Assisted-by: Codex[GPT-5]
```

Your branch is ahead of `origin/main` by 2 commits.

USER

Maybe for the status field when creating a task, remove it and have it default to “To do”, then people can just edit its status

ASSISTANT

I'll remove the status selector from the create form only. The database/service value should remain `Todo` to match the rubric's allowed status names, and users can still change it later through Edit.

ASSISTANT

I'm also adjusting one test so the default-status behavior is explicitly covered: creating a task without a status should persist as `Todo`. That protects the new workflow.

ASSISTANT

The create form no longer asks for status, and the service test now verifies the default `Todo` status when omitted. I'm running tests and build now.

ASSISTANT

Verification passed. I'm committing this focused form change and test adjustment with the required AI trailer.

ASSISTANT

The commit is done. I'm checking the final branch state now.

ASSISTANT

Done. The create form no longer shows the status field. New tasks now default to `Todo`, and users can change the status later through Edit.

I kept the stored status as `Todo` rather than `To do` because the rubric/database allowed values are exactly `Todo`, `In-Progress`, and `Complete`.

Verified:

```
npm test
npm run build
```

Committed as:

```
4309535 Default new tasks to todo status
Assisted-by: Codex[GPT-5]
```

USER

Format title field to be double the length and then put topic field next to due date field so it looks more organized

ASSISTANT

I'll adjust the create form layout so the title spans the full row, then topic and due date sit together on the next row. That keeps the data model untouched and only changes the visual arrangement.

ASSISTANT

I'm adding a targeted class for the create form title, so edit forms keep their existing compact layout.

ASSISTANT

The layout change is in place: title spans both columns, and topic/due date share the next row. I'm running tests and build before committing.

ASSISTANT

Verification passed. I'm committing the small layout tweak now with the required trailer.

ASSISTANT

One earlier local commit is still unpushed; this new commit will add to that ahead count.

ASSISTANT

Done. The create form now lays out as:

- Title across the full row
- Topic and due date next to each other underneath
- Description below as before

Verified:

```
npm test
npm run build
```

Committed as:

```
b7ddc35 Widen create task title field
Assisted-by: Codex[GPT-5]
```

USER

One small bug, when you update the status, it does successfully update the status but the preview to the edit status field defaults back to “To do” until you refresh the page, then it updates to its actual status

ASSISTANT

I'll fix this in the edit form, not the data layer. This sounds like an uncontrolled `<select defaultValue=...>` staying stale after the server action, so the reliable fix is to remount the edit form fields whenever the persisted task values change.

ASSISTANT

The edit form currently uses `defaultValue` fields, which are only applied when the form first mounts. I'm keying the edit form by the task's persisted update timestamp so after a save, the browser gets a fresh form reflecting the saved values.

ASSISTANT

The remount fix is in place. I'm running the test suite and production build now to make sure this client-component change still compiles cleanly.

ASSISTANT

Tests and build passed. I'm committing this bug fix now.

ASSISTANT

Fixed. The edit form now remounts after a task update using the task's `updatedAt` value, so the status dropdown should immediately reflect the saved status instead of showing the old/default value until refresh.

Verified:

```
npm test
npm run build
```

Committed as:

```
0c03847 Refresh edit form after task updates
Assisted-by: Codex[GPT-5]
```

Test it by editing a task from `Todo` to `In-Progress` or `Complete`, saving, then reopening/looking at the edit form without refreshing.

USER

Would you be able to improve the editing format so instead of having a whole new thing under, maybe have the Existing fields be editable with a box appearing to show its editable. For the status, maybe have all 3 status appear with the ones active be colored while the others arent and if u click on the gray ones, it switches to that

ASSISTANT

I'll replace the expandable edit section with an inline edit mode on the card itself. The card will switch its displayed fields into inputs, and the status will become three segmented radio options where the selected one is highlighted.

ASSISTANT

I'm changing the task card structure now: view mode will show the normal card, and pressing Edit will transform that same card into an inline form. The status picker will be radio-based so the selected option changes visually immediately before saving.

ASSISTANT

The JSX is now switched to inline edit mode. I'm tightening the markup and CSS so the edit state looks intentional rather than like a separate form dropped into the card.

ASSISTANT

I'll update the CSS now: editable cards get a stronger border/glow, inputs sit exactly where the display content normally sits, and the status choices behave like a segmented control.

ASSISTANT

The inline edit styling is in place. I'm running tests and build now; then I'll start the dev server again if needed so you can inspect the interaction.

ASSISTANT

One lifecycle detail needs tightening: after a second edit, the old success state can otherwise prevent the card from closing again. I'm tying the close behavior to the persisted `updatedAt` value so each successful save behaves consistently.

ASSISTANT

I'm adding one responsive rule so the inline editor stacks cleanly on narrow screens instead of squeezing the status options.

ASSISTANT

The responsive polish is done. I'm running the usual verification now, then I'll commit the inline edit redesign.

ASSISTANT

Tests and build both passed. I'm committing the edit UI redesign now.

ASSISTANT

You have three local commits already ahead of GitHub; this edit redesign will make it four until you push.

ASSISTANT

The commit is done. I'm checking the final tree so I can close with the exact state.

ASSISTANT

Implemented the inline edit redesign.

Now when you click **Edit**, the task card itself becomes editable: - Title, description, topic, and due date turn into input fields in place. - Status shows as three clickable options: **Todo**, **In-Progress**, **Complete**. - The active status is colored; inactive statuses stay muted/gray. - Clicking another status immediately switches the visual selection before saving. - **Save changes** persists it, and **Cancel** exits edit mode.

Verified:

```
npm test
npm run build
```

Committed as:

```
ebdcea4 Redesign task editing inline
Assisted-by: Codex[GPT-5]
```

Your branch is now ahead of `origin/main` by 4 commits.

USER

Update the README if it needs any updates and do any final checks against the rubric to make sure its all fine for the highest marks

ASSISTANT

I'll do this as a final submission pass: check README against the PDF requirements, verify commits/AI trailers, run tests/build, and make small README updates only where the current app behavior changed enough to document.

ASSISTANT

The README has the required sections, but it is missing a short note about the current UI behavior after the recent changes. I'm going to add that, update the testing bullets to match the default-status workflow, and add a reminder that the AI transcript is submitted separately.

ASSISTANT

README is updated. I'm running the final verification commands and checking the evidence items the marker is likely to inspect: clean tracked files, commit count, trailers, migrations, tests, and build.

ASSISTANT

All verification commands passed. I'm committing the README update, then I'll do one final rubric-oriented status check.

ASSISTANT

The README update is committed. I'm doing the last clean-tree and rubric evidence check now.

ASSISTANT

Final pass is done. I updated the README to reflect the current app behavior: default `Todo` status, inline editing, archived-task toggle, dark-mode default, and separate AI transcript submission.

Committed as:

```
16e2f0b Update README for final app behavior
Assisted-by: Codex[GPT-5]
```

Verification passed:


```
npm run db:migrate
npm test
npm run build
```

Rubric check looks strong:

- **Running docs:** present in README.md.
- **Third-party code docs:** present.
- **Database design docs:** present and detailed.
- **Student ID/name:** present.
- **SQLite persistence:** implemented with `data/todo.db`.
- **Create/edit/archive tasks:** implemented.
- **Required fields:** title, description, due date, topic, status.
- **Statuses:** exactly `Todo`, `In-Progress`, `Complete`.
- **Overdue:** derived correctly, not treated as a status.
- **Tests:** 5 deterministic Vitest tests, including archive and overdue.
- **Commits:** 22 commits, comfortably above the required 6.
- **AI declaration:** present in README.
- **Transcript reminder:** now documented in README.

One residual policy risk: old commit `2b993e3` `Small change to documentation` has no **Assisted-by** trailer. All recent Codex-assisted commits do have the trailer. I would not rewrite history unless your marker is extremely strict about every historical commit.

Current branch is clean and ahead of GitHub by 1 commit. Push before submission:

```
git push
```

USER

Remove the Repo note cause it doesn't make sense to put the repo in the README if they're already on the repo side. Also remove that WSL note, and also remove the 3001 notes. Keep the "Running it" Section simple so only give the basics on starting it

ASSISTANT

I'll simplify **Running It** and remove the repository URL, alternate port, and WSL troubleshooting notes. I'll keep only the clean-clone basics plus test/build commands because those are useful for marking.

ASSISTANT

I'm editing only the README. The simplified section will keep `npm install`, migration, dev server, localhost URL, tests, and build.

ASSISTANT

The README is simplified. I'm checking the diff, then I'll commit it with the AI trailer.

ASSISTANT

This is docs-only, so I'm not rerunning the build. I'll commit the README cleanup now.

ASSISTANT

Done. I removed:

- The repository URL line
- The 3001 alternate port instructions
- The WSL troubleshooting note
- Extra production/migration regeneration notes that made `Running It` noisy

The `Running It` section now just has the basics: install, migrate, run dev server, open `localhost:3000`, run tests, and build.

Committed as:

a0ba0eb Simplify running instructions

Assisted-by: Codex[GPT-5]

Your branch is ahead of GitHub by 1 commit, so push when ready.

USER

Actually how about this, make a doc folder and put a “Running it” markdown file giving a detailed explanation on using, but keep the brief one the readme. Have a separate “dependencies” markdown with the third-party code details so remove it from the readme. Likewise with the database design. Keep the Testing and AI Usage Declaration part on the read me

ASSISTANT

I'll split the bulky documentation into `docs/` while keeping the README as the short submission front page. README will keep identity, brief running steps, testing, and AI usage; detailed running/dependencies/database sections move into separate Markdown files.

ASSISTANT

I'm keeping the exact rubric headings available in Markdown, just moved out of the README. The README will link to `docs/running-it.md`,

`docs/dependencies.md`, and `docs/database-design.md` so a marker can still find them immediately.

ASSISTANT

The docs are split now. I'm checking the README and new docs for structure, then I'll run tests/build to make sure this final docs-only change doesn't hide any accidental file issue.

ASSISTANT

The split looks clean. I'm running the final verification commands now, then I'll commit the documentation restructure.

ASSISTANT

Verification passed. I'm making the README links clickable, then I'll stage the README plus the three new docs files.

ASSISTANT

The README now links directly to the detailed docs. I'm committing the documentation split with the required AI trailer.

ASSISTANT

The documentation split is exactly scoped: README plus three new Markdown files. I'll stage and commit now.

ASSISTANT

The docs restructure is committed. I'm doing a final clean-tree check.

ASSISTANT

Done. I split the documentation like this:

- README keeps the brief **Running It, Testing, and AI Usage Declaration**.
- Detailed running instructions moved to `docs/running-it.md`.
- Third-party code/dependencies moved to `docs/dependencies.md`.
- Database design moved to `docs/database-design.md`.

Verified before committing:

```
npm test
npm run build
```

Committed as:

af8fb84 Move detailed documentation into docs
Assisted-by: Codex[GPT-5]

Your branch is ahead of GitHub by 1 commit, so push when ready.

USER

for all the markdowns you made, indicate it was generated by you like with README. For the detailed running it one, have it be more detailed starting with “git clone ...” and cd, etc, also include troubleshooting notes

ASSISTANT

I'll update all three docs files with the same AI declaration style used in the README. I'll also expand docs/running-it.md into a full clean-clone walk-through starting with `git clone`, `cd`, `install`, `migrate`, `run`, `verify`, and `troubleshooting`.

ASSISTANT

I'm replacing the running guide with a fuller end-to-end version and appending AI declarations to each docs file. The commands will stay copy-pasteable and avoid local machine assumptions except where clearly marked as troubleshooting.

ASSISTANT

The docs now include AI declarations, and the running guide starts from `git clone`. I'm reviewing the final text and running tests/build before committing.

ASSISTANT

The detailed guide now includes troubleshooting notes. I'm running the final verification commands and then I'll commit the documentation updates.

ASSISTANT

Tests and build passed. I'm committing the docs updates now with the AI trailer.

ASSISTANT

The commit is done. I'm checking final status.

ASSISTANT

Done.

Updated: - docs/running-it.md now starts with `git clone`, `cd`, `install`, `migrate`, `run`, `verification` commands, `database` commands, and `troubleshooting` notes. - docs/dependencies.md now includes an AI declaration. - docs/database-design.md now includes an AI declaration.

Verified:

```
npm test
npm run build
```

Committed as:

```
0204250 Expand detailed documentation
Assisted-by: Codex[GPT-5]
```

Branch is ahead of GitHub by 1 commit.

USER

In the README, remove this part " The app runs locally for a single user and stores tasks in data/todo.db. New tasks start as Todo; use inline editing to update task details and move tasks between Todo, In-Progress, and Complete. Archived tasks are kept out of the main list until the View Archived Tasks control is opened. The interface defaults to dark mode and remembers the selected appearance mode in the browser.", instead make a features heading and organize it there. Make the Documentation part more organized by just having be hyperlinks with Organized names

ASSISTANT

I'll reorganize the README intro into a short description plus a **Features** section, and make the docs list use cleaner link names. This is a README-only change.

ASSISTANT

I'm moving those behavior details into a concise feature list and simplifying the docs links to names a marker can scan quickly.

ASSISTANT

The README structure is updated. I'll review the diff and commit it with the required trailer.

ASSISTANT

This is docs-only, so no build run is needed. I'm committing the README cleanup now.

ASSISTANT

The README update is committed. I'm checking final status.

ASSISTANT

Done. README now has:

- A short intro
- A separate **Features** section
- Cleaner documentation links:
 - **Running Guide**
 - **Dependencies**
 - **Database Design**

Committed as:

0e49e2e Organize README feature summary

Assisted-by: Codex[GPT-5]

Branch is ahead of GitHub by 1 commit.